



Projeto Prático 2 – Árvores Multi-caminhos

Diretrizes para o projeto prático com Árvores Multi-caminhos.

1. Equipes com, **no máximo**, 3 integrantes.
2. Utilizar a implementação disponibilizada no moodle neste link:
“Framework de Persistência com índice (Árvore B+)”
<https://moodle.utfpr.edu.br/mod/resource/view.php?id=2178366>
3. O que a implementação disponibilizada contém?
 - a. a biblioteca de persistência (construída no semestre passado) incrementada com um sistema de índice baseado em árvore B+.
 - b. a biblioteca de árvore B+.
 - c. o benchmark gerado pela ferramenta de IA que avaliou a performance das operações da biblioteca de persistência com indexação e sem indexação.
 - d. o relatório em PDF descrevendo os resultados e conclusões do benchmark.
4. O relatório mostra que a operação *deletee* com índice fica catastroficamente mais lento que sem índice à medida que N cresce (**7,85s contra 0,18s em N=100.000**) — e a conclusão do relatório já aponta a causa e até sugere a correção como trabalho futuro: em *Operacoes/5_delete.h*, cada remoção reescreve o arquivo de dados inteiro (*porque os offsets dos sobreviventes mudam*) e, pior, joga fora a árvore B+ inteira e a reconstrói do zero, reinserindo um a um todos os registros restantes — em vez de usar o *removeArv* que a própria biblioteca *ArvoreBMais* já implementa corretamente, com empréstimo entre irmãos e fusão de nós.
 - a. Implemente uma versão de *deletee* que use *removeArv* de forma incremental, sem reconstruir a árvore. Isso deve resolver o problema real por trás do gargalo — o *offset* do registro removido não pode simplesmente sumir do arquivo sem invalidar os *offsets* guardados no índice para todo mundo depois dele.
 - b. Duas soluções clássicas:
 - i. **(1)** exclusão lógica com *"tombstone"* (marca o registro como removido sem deslocar os demais, e só compacta o arquivo periodicamente) ou
 - ii. **(2)** *swap* com o último registro do arquivo (troca o registro a remover pelo último, trunca o arquivo em 1 registro, remove a chave antiga do índice com *removeArv* e atualiza a entrada do registro movido). Execute o arquivo *bench.c* novamente (ele já existe e já gera o CSV comparativo) e

verifique empiricamente se a curva de *deletee* deixou de crescer em $O(n \log n)$ e passou a acompanhar o crescimento de *createe/retrieve*.

5. Outro aspecto a ser explorado é variação do parâmetro ORDEM_INDICE (atualmente fixo em 4, em *Operacoes/0_structs.h*) e observar o efeito no desempenho e na altura da árvore. Como a árvore não expõe altura nem contagem de nós, faz-se necessário instrumentar a operação *criarNoh/inserirArv* para contar *splits* e calcular a altura, rodar o benchmark para diferentes ordens como 4, 8, 16, 32, 64 e discutir o *trade-off* clássico da árvore B+: *uma ordem maior reduz a altura (menos níveis para descer), mas aumenta o custo de busca dentro de cada nó, já que a operação **encontrarFolha** faz uma busca linear no vetor de chaves*.
6. De acordo com o relatório, o desempenho da operação *queryAll* sem índice foi muito superior da versão com índice (*conforme gráfico na página 3*). Analisar criticamente esse desempenho considerando que a árvore B+ possui uma lista com todos os registros do arquivo, o que deveria otimizar a operação *queryAll* (listagem completa dos dados cadastrados).
7. Permitido o uso de ferramentas de IA para a implementação e avaliação do sistema de índice para sistema de arquivos baseado em árvore B (ou B+).
8. A equipe deverá entregar um **relatório descrevendo os resultados da avaliação dos itens 4, 5 e 6 apontados acima**. Importante destacar as conclusões a respeito dos resultados obtidos.
9. Enviar o relatório do benchmark na tarefa do moodle denominada “**Projeto Prático 2 – Árvore B (B+)**” até a data limite **04/10/2026**.